

BÁO CÁO BÀI TẬP LỚN | LẬP TRÌNH NÂNG CAO

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Auction System - Online Bidding Platform

Giảng viên: [Vũ Trọng Thanh - Phạm Tùng Thủy - Kiều Văn Tuyên]

Sinh viên thực hiện:

- Nguyễn Hữu Hùng
- Đặng Đức Anh
- Tô Bảo Hân
- Bùi Thế Dũng

Công nghệ cốt lõi: Java, JavaFX, Socket TCP, MySQL, Maven
Hà Nội, Phiên bản tài liệu 1.0 — Tháng 5 / 2026

Mục lục

Tóm tắt điều hành (Executive Summary)	2
1 Giới thiệu mục tiêu và phạm vi	2
1.1 Mục tiêu dự án	2
1.2 Phạm vi chức năng	2
2 Kiến trúc tổng thể của hệ thống	2
2.1 Mô hình kết nối Client-Server	2
2.2 Cấu trúc phân tầng hệ thống	3
2.3 Minh họa thiết kế định tuyến thông điệp tại Server	3
3 Chi tiết các chức năng đạt được	3
3.1 Thiết kế lớp và cây kế thừa ứng dụng nguyên lý OOP	3
3.2 Mẫu thiết kế áp dụng thực tế (Design Patterns)	4
3.3 Xử lý đấu giá đồng thời (Concurrent Bidding)	4
3.4 Các thuật toán nâng cao bổ sung (Tính điểm thưởng)	4
3.5 Đảm bảo chất lượng phần mềm & CI/CD Pipeline	5
4 Kết luận và Hướng phát triển	5

Tóm tắt điều hành (Executive Summary)

Báo cáo này trình bày kết quả thiết kế, kiến trúc và triển khai thực tế dự án **Hệ thống Đấu giá Trực tuyến**. Bằng việc ứng dụng mô hình Client-Server đa luồng tách biệt thông qua giao tiếp Socket TCP và cấu trúc dữ liệu JSON, hệ thống cho phép người dùng tham gia tương tác đấu giá thời gian thực một cách toàn vẹn. Trọng tâm của báo cáo tập trung làm rõ việc áp dụng các nguyên lý lập trình hướng đối tượng (OOP) nâng cao, xử lý đồng thời chuyên sâu nhằm loại bỏ hoàn toàn tranh chấp dữ liệu, cùng các thuật toán tối ưu trải nghiệm và bảo mật hệ thống.

1. Giới thiệu mục tiêu và phạm vi

1.1 Mục tiêu dự án

- **Làm chủ tư duy OOP nâng cao:** Hiện thực hóa các nguyên lý Đóng gói (*Encapsulation*), Kế thừa (*Inheritance*), Đa hình (*Polymorphism*) và Trừu tượng (*Abstraction*) vào một hệ thống phần mềm có quy mô lớn.
- **Kiến trúc Client-Server chuẩn mực:** Triển khai phân tầng MVC, xử lý thông điệp mạng dựa trên sự kiện (Action-based messaging) bất đồng bộ.
- **Xử lý đồng thời (Concurrency):** Loại bỏ lỗi *race condition* và nguy cơ ghi đè dữ liệu sai lệch khi có nhiều tài khoản cùng thực hiện trả giá tại cùng một thời điểm.

1.2 Phạm vi chức năng

Hệ thống phân định quyền hạn rõ ràng thông qua kiến trúc phân quyền 3 lớp người dùng:

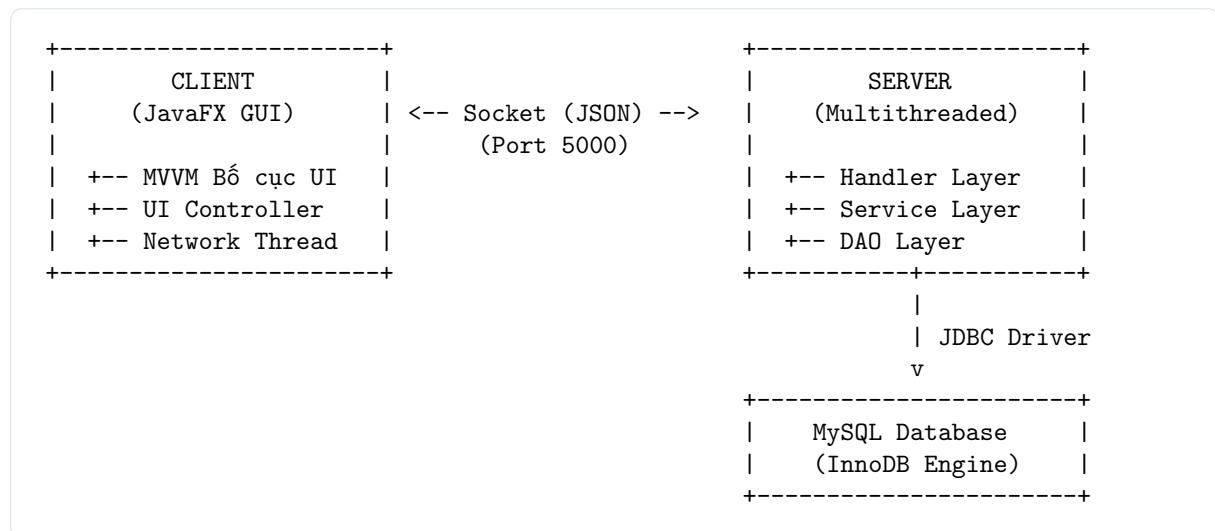
- **Bidder (Người mua):** Xác thực bảo mật, duyệt danh sách phòng đấu giá, thực hiện đặt giá thời gian thực, xem biểu đồ biến động giá và nhận thông báo đẩy tức thời.
- **Seller (Người bán):** Tạo phiên đấu giá mới, kiểm soát giá sàn, bước nhảy giá và quản lý vòng đời sản phẩm.
- **Admin (Quản trị viên):** Xét duyệt trạng thái phiên đấu giá, đóng hoặc hủy các phiên vi phạm chính sách của hệ thống.

Vòng đời trạng thái phiên được quản lý nghiêm ngặt qua các bước: OPEN → RUNNING → SOLD / UNSOLD / CLOSED_BY_SELLER / CANCELED_BY_ADMIN.

2. Kiến trúc tổng thể của hệ thống

2.1 Mô hình kết nối Client-Server

Giao tiếp mạng giữa ứng dụng JavaFX Client và Multithreaded Server sử dụng luồng dữ liệu Socket chuyên biệt qua cổng dịch vụ 5000:



2.2 Cấu trúc phân tầng hệ thống

- **Phía Client:** Gồm tầng UI (FXML/CSS) để hiển thị giao diện, tầng Controller điều hướng sự kiện, và tầng Network duy trì kết nối luồng Socket ngầm để nhận dữ liệu liên tục.
- **Phía Server:** Handler Layer chịu trách nhiệm giải mã gói tin JSON và định tuyến thông điệp; Service Layer thực thi logic nghiệp vụ cốt lõi; DAO Layer kết nối, truy vấn dữ liệu an toàn.

2.3 Minh họa thiết kế định tuyến thông điệp tại Server

```

1 public class BidHandler {
2     private final AuctionService auctionService;
3
4     public void handleBidMessage(BidRequest req) {
5         if (!validateBid(req)) return; // Kiểm tra tính hợp lệ đầu vào
6
7         // Gọi tầng Service thực hiện khoa phiên đóng thời an toàn
8         auctionService.processBid(req.auctionId, req.bidderId, req.
amount);
9
10        // Broadcast cập nhật dữ liệu mới đến tất cả các Client
11        broadcastUpdate(req.auctionId);
12    }
13 }
    
```

3. Chi tiết các chức năng đạt được

3.1 Thiết kế lớp và cây kế thừa ứng dụng nguyên lý OOP

Hệ thống tổ chức cấu trúc lớp sâu sắc nhằm triệt tiêu việc trùng lặp mã nguồn:

- **Mô hình hóa thực thể:** Lớp trừu tượng User làm gốc cho Bidder, Seller, Admin. Lớp trừu tượng Item làm gốc cho các phân loại hàng hóa cụ thể như Electronics, Art.
- **Tính Đa hình & Trừu tượng:** Triển khai đa hình thông qua việc ép các lớp con ghi đè phương thức hiển thị `getDisplayInfo()` riêng biệt và xử lý dòng tiền kết thúc phiên đấu giá qua hàm `processAuctionCompletion()`. Mật khẩu người dùng được bao bọc đóng gói và mã hóa an toàn bằng thuật toán mã băm **BCrypt(12)**.

3.2 Mẫu thiết kế áp dụng thực tế (Design Patterns)

- **Singleton Pattern:** Áp dụng cho lớp `DatabaseConnection` tích hợp bộ thư viện quản lý kết nối HikariCP giúp tối ưu hóa tài nguyên kết nối, chống rò rỉ bộ nhớ.
- **Factory Method Pattern:** Sử dụng `ItemFactory` để đóng gói logic khởi tạo các đối tượng sản phẩm phức tạp dựa trên các tham số đầu vào động.
- **Observer Pattern:** Xây dựng cơ chế đăng ký và lắng nghe sự kiện đẩy thông tin giá mới, giúp cập nhật trạng thái phòng đấu giá tức thời.

3.3 Xử lý đấu giá đồng thời (Concurrent Bidding)

Bài toán đặt ra: Ngăn chặn tình trạng hai người dùng cùng đặt một mức giá ở cùng một thời điểm gây ra hiện tượng xung đột ghi dữ liệu (*Lost Update*).

Giải pháp: Sử dụng cơ chế khóa tường minh `ReentrantLock` tích hợp trong cấu hình trạng thái của từng phiên đấu giá cụ thể:

```
1 public class AuctionRuntimeState {
2     private final ReentrantLock lock = new ReentrantLock();
3     private double currentPrice;
4
5     public void placeNewBid(String bidder, double amount) {
6         lock.lock(); // Kích hoạt doc quyen vao vung tranh chap
7         try {
8             if (amount <= currentPrice) {
9                 throw new BidTooLowException();
10            }
11            currentPrice = amount;
12            this.highestBidder = bidder;
13        } finally {
14            lock.unlock(); // Dam bao luon luon giai phong khoa
15        }
16    }
17 }
```

3.4 Các thuật toán nâng cao bổ sung (Tính điểm thưởng)

- **Anti-sniping Algorithm (Chống bắn tỉa phút chót):** Nhằm mục đích giữ tính minh bạch cho sàn đấu giá, trong hàm xử lý đặt giá, nếu thời gian còn lại của phiên nhỏ hơn 30 giây,

hệ thống tự động gọi hàm `extendBySeconds(60)` để gia hạn thêm 1 phút đồng thời đẩy gói tin `EXTENSION_NOTIFY` về phía giao diện Client.

- **Auto-Bidding (Đấu giá tự động):** Thiết lập cấu trúc hàng đợi ưu tiên `PriorityQueue` lưu trữ các yêu cầu tự động dựa trên mức giá trần `maxBid`. Server sẽ tính toán bước nhảy để tự động nâng giá đại diện cho người mua.
- **Biểu đồ lịch sử giá thời gian thực (Price Curve Visualization):** Sử dụng thành phần đồ thị `LineChart` của `JavaFX`. Khi luồng mạng `Socket` nhận được gói tin định dạng `UPDATE_PRICE`, dữ liệu mới sẽ được nạp trực tiếp vào mảng tọa độ đồ thị mà không cần tải lại trang.

3.5 Đảm bảo chất lượng phần mềm & CI/CD Pipeline

- Toàn bộ cấu trúc mã nguồn được định dạng theo quy chuẩn sạch *Google Java Style Guide*.
- Logic nghiệp vụ tính toán dòng tiền, kiểm thử điều kiện đặt giá đều được bao phủ bởi các kịch bản `Unit Test` nghiêm ngặt sử dụng thư viện **JUnit 5** và **Mockito**.
- Hệ thống tích hợp thành công luồng tự động hóa **GitHub Actions** để biên dịch và chạy kiểm thử tự động mỗi khi có thành viên đẩy mã nguồn mới lên hệ thống quản lý mã nguồn.

```
1 name: Automated Testing Pipeline
2 on: [push, pull_request]
3 jobs:
4   test-suite:
5     runs-on: ubuntu-latest
6     steps:
7       - uses: actions/checkout@v3
8       - uses: actions/setup-java@v3
9         with: { java-version: '17', distribution: 'temurin' }
10      - name: Execute Automated Unit Tests
11        run: mvn clean test
```

Tập cấu hình `.github/workflows/test.yml`

4. Kết luận và Hướng phát triển

Báo cáo của nhóm 4 thành viên (**Nguyễn Hữu Hùng, Đặng Đức Anh, Tô Bảo Hân, Bùi Thế Dũng**) đã làm rõ việc hoàn thành trọn vẹn các yêu cầu bài tập lớn đặt ra. Hệ thống đạt độ ổn định cao nhờ thiết kế hướng đối tượng mạch lạc, giải quyết triệt để bài toán tranh chấp luồng bằng `ReentrantLock` và cơ chế truyền tải thông điệp hướng sự kiện thời gian thực.

Trong tương lai, nhóm định hướng sẽ nâng cấp từ cấu trúc `Socket` thuần sang giao thức **WebSocket** để tăng tính tương thích nền tảng, tích hợp bộ nhớ đệm **Redis Caching** để giảm tải truy vấn cho Database trung tâm, đảm bảo hệ thống có khả năng mở rộng phục vụ hàng vạn kết nối đồng thời.